第 6 天 · 11/7 · 今日知识点回顾

面向对象 + 模块与包 + 常用标准库

目标：写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 73 分钟（边看边在本机敲一遍）

P111	初识对象	约 10 分钟
P112	类的成员方法	约 18 分钟
P114	构造方法	约 12 分钟
P115	魔术方法	约 15 分钟
P94	模块的概念和导入	约 18 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/18.面向对象编程入门.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西

- 一句话：class 定义模板，__init__ 在造对象时自动跑，self 指「这个对象自己」。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id = order_idself.city = cityself.amount = amount

这一页只讲「类 = 模板，对象 = 按模板造出来的东西」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西（续）

```
o = Order('100001', '北京', 1899.0)
print(o.city)          # 北京
print(o.amount)        # 1899.0
```

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

知识点 2/7 • __repr__ 决定「print 出来长什么样」

- 一句话：没写 __repr__ 时 print(o) 打印 <__main__.Order object at 0x...>，看不出内容。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

这一页只讲「__repr__ 决定「print 出来长什么样」」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • __repr__ 决定「print 出来长什么样」（续）

```
def __repr__(self):
return f'Order({self.order_id}, {self.city}, {self.amount:.2f})'
print(Order('100001', '北京', 1899.0))     # Order(100001, 北京, 1899.00)
```

★ 调试时 90% 的时间花在「看对象里有什么」。__repr__ 写好了，print 一次就够；

- 不写，就得 print(o.__dict__) 或进调试器。

知识点 3/7 • 方法：把「针对这个对象的操作」写进类里

- 一句话：方法就是第一个参数是 self 的函数，能直接访问 self.xxx。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

知识点 3/7 • 方法：把「针对这个对象的操作」写进类里（续）

```
def is_big(self, threshold=1000):
return self.amount >= threshold
def discounted(self, rate=0.9):
return round(self.amount * rate, 2)
o = Order('100001', '北京', 1899.0)
print(o.is_big(), o.discounted())      # True 1709.1
```

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

知识点 4/7 • 继承：把公共部分提到父类

- 一句话：class 子类(父类): 自动拥有父类的属性和方法，需要改就覆写。
- 代码：

```
class Base:
def describe(self):
return f'{type(self).__name__}: {self.__dict__}'
class PaidOrder(Base):
def __init__(self, oid, amount):
```

知识点 4/7 • 继承：把公共部分提到父类（续）

- self.oid, self.amount = oid, amount

```
print(PaidOrder('100001', 1899.0).describe())
```

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

- 真正高频的是「一个类装一份数据 + 几个方法算指标」，今天写出来就够。

知识点 5/7 • import 的三种写法，差别在「名字进不进当前命名空间」

- 一句话：import x / from x import y / import x as z，优先用第一种或第三种。
- 代码：

```
import pathlib                  # 用 pathlib.Path
from pathlib import Path        # 直接用 Path
import numpy as np              # 惯例缩写
import pandas as pd
```

★ from module import * 会把模块里所有公开名字倒进来，容易和你的变量撞名，而且看不出某个函数是哪来的 —— 项目里禁止用（PEP 8 明确不建议）。

知识点 6/7 • 四个天天要用的标准库

- 一句话：pathlib 管路径、datetime 管日期、collections 管计数、json 管序列化。
- 代码：

```
import pathlib, datetime as dt, collections, json
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
print(DATA.exists(), (DATA / 'orders_dirty.csv').stat().st_size)
print(dt.date(2026, 11, 7).strftime('%Y-%m-%d %A'))
c = collections.Counter(['北京', '上海', '北京'])
print(c.most_common())            # [('北京', 2), ('上海', 1)]
print(json.dumps({'city': '北京'}, ensure_ascii=False))   # {"city": "北京"}
```

★ json.dumps 默认 ensure_ascii=True，中文会变成 \u5317\u4eac。写文件给人看时一定要 ensure_ascii=False，否则打开是一堆转义。

知识点 7/7 • 把函数收进类：什么时候值得

- 一句话：一份数据要反复算多个指标、或要带着状态（清洗前后的行数）时，才值得建类。
- 代码：

```
class Orders:
def __init__(self, rows):
```

- self.rows = rowsself.dropped = 0

```
def filter_paid(self):
before = len(self.rows)
```

知识点 7/7 • 把函数收进类：什么时候值得（续）

- self.rows = [r for r in self.rows if r['status'].strip().lower() == 'paid']self.dropped += before - len(self.rows)

```
return self
```

（self.dropped 这种「顺手记一笔」的状态，是类比纯函数好用的地方：

- 纯函数每次都要把计数传进去再传出来。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 5 天的四个函数收进一个类，并加两个方法：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
class Orders:
```

```
    def __init__(self, rows):
        self.rows = rows
        self.log = []
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

def __repr__(self):

```
        return f'Orders({len(self.rows)} 行，已丢弃 {sum(self.log) if self.log else 0})'
```

```
    def filter_status(self, status='paid'):
```

before = len(self.rows)

```
        self.rows = [r for r in self.rows if r['status'].strip().lower() == status]
```

```
        self.log.append(before - len(self.rows))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
        return self
    def sum_by(self, col='city'):
        agg = {}
        for r in self.rows:
            k = r[col] or '（未知）'
```

```
            v = float(r['amount'].replace('¥', '').replace(',', '') or 0)
```

```
            agg[k] = agg.get(k, 0.0) + v
```

```
        return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
```

```
    rows = list(csv.DictReader(f))
o = Orders(rows)
print(o)
print(o.filter_status('paid'))
top = o.sum_by('city')
for k, v in list(top.items())[:3]:
    print(f'{k}\t{v:,.2f}')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

验收（做到才算过关）：

```
① print(o) 打印的是 Orders(1200 行, 已丢弃 0)，不是 <object at 0x...>；
```

② filter_status 之后 print(o) 能看出丢了多少行；

```
③ 能说出「为什么 sum_by 里对 amount 又要洗一遍」— 因为第 3 天没洗，
```

原始 rows 里的 amount 还带着 ¥ 和逗号。

最容易错的地方

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 7 天 • 文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-07-第06天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data